ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ГОРОДА МОСКВЫ
«КОЛЛЕДЖ СВЯЗИ №54»
ИМЕНИ П.М. ВОСТРУХИНА

ПРАКТИЧЕСКАЯ РАБОТА № 2

МДК.05.03

Тестирование информационных систем

Тема: «Bash: базовые проверки ИС и обработка сбоев»

Выполнила

ст. Гр. 4-ИСП9-46

Миронова Анна Андреевна

Москва, 2026

СОДЕРЖАНИЕ

Цель работы 3

Выполненная работа 3

Средства разработки 3

Анализ сложности 3

Результаты выполнения 4

Ответы на контрольные вопросы 5

Проверка результатов 6

Заключение 7

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ И ИНТЕРНЕТ-ИСТОЧНИКОВ 8

Листинги программ 9

Цель работы

Проверить корректность работы ИС и научиться обнаруживать воспроизводимые дефекты.

Условие: practice_rosa_linux_basic_test_kernel.md.md.

Выполненная работа

Реализован run_tests.sh с проверками CSV-файла, чтения, числа строк, структуры, процесса, порта, HTTP-кода и содержимого. Добавлены порог ответа 1 секунда, уникальные ID и alerts.log на каждый FAIL. Агрегатор корректно возвращает exit 1, даже если последний отдельный тест прошёл.

Средства разработки

Python 3.14.6, Flask 3.1.2, SQLite 3.53.3. Проверки выполнены в Arch Linux/WSL. Запуск в POCA и Python 3.8 отдельно не проверялся.

Порядок запуска: инструкция.

Анализ сложности

Чтение CSV и HTTP-тела O(B); поиск дубликатов через sort O(n log n), память зависит от sort. Набор из k последовательных проверок O(k) обращений с ограниченными таймаутами.

Самостоятельная работа

Проверка response_time требует успешного curl и time_total < 1. Порог проверяется по фактическому времени, а не по HTTP-коду. test_unique_ids.sh отклоняет повтор ID. check.sh пишет FAIL в alerts.log и диагностический журнал. Cron подготовлен для запуска каждые 5 минут; фактические события приведены в отдельном приложении. У сервера, остановленного после положительного прогона, сетевые проверки возвращают FAIL, а файловые — PASS.

Результаты выполнения

Результат проверки показан на рисунке 1.

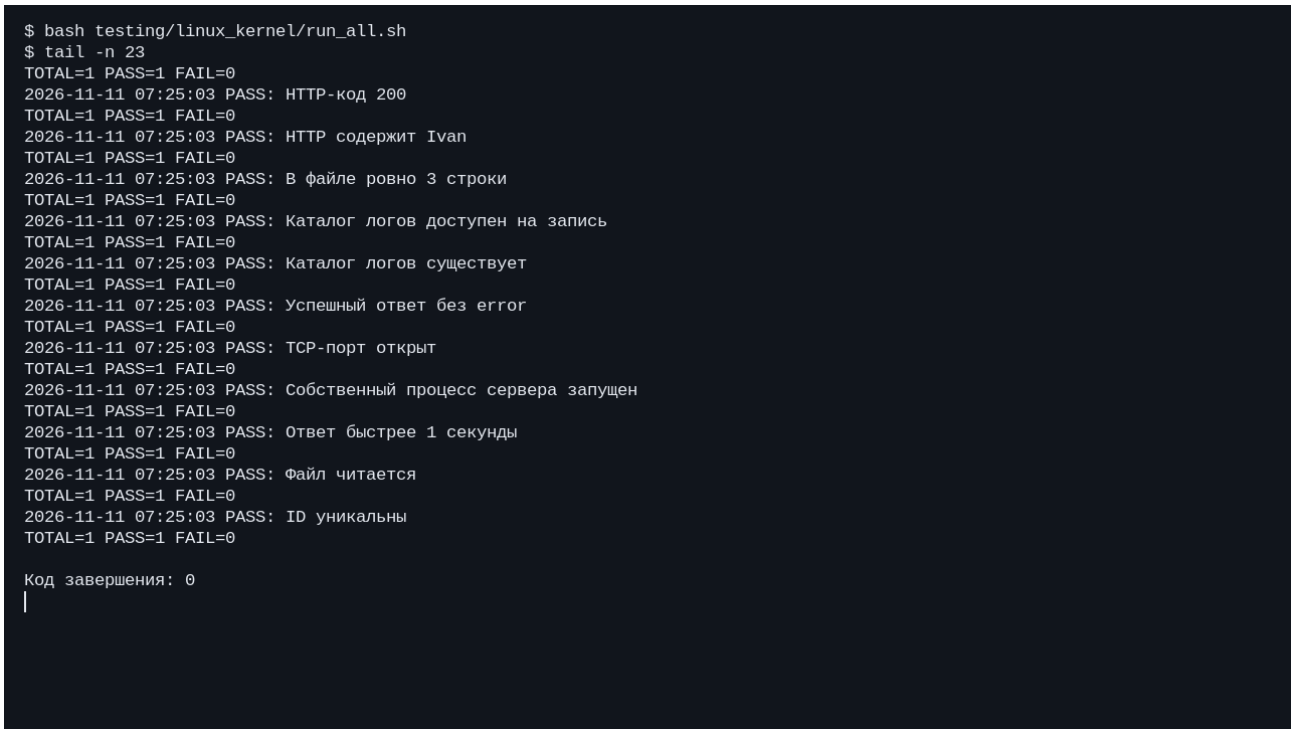


Рисунок 1 – Успешные автоматические проверки работающей системы

Результат проверки показан на рисунке 2.

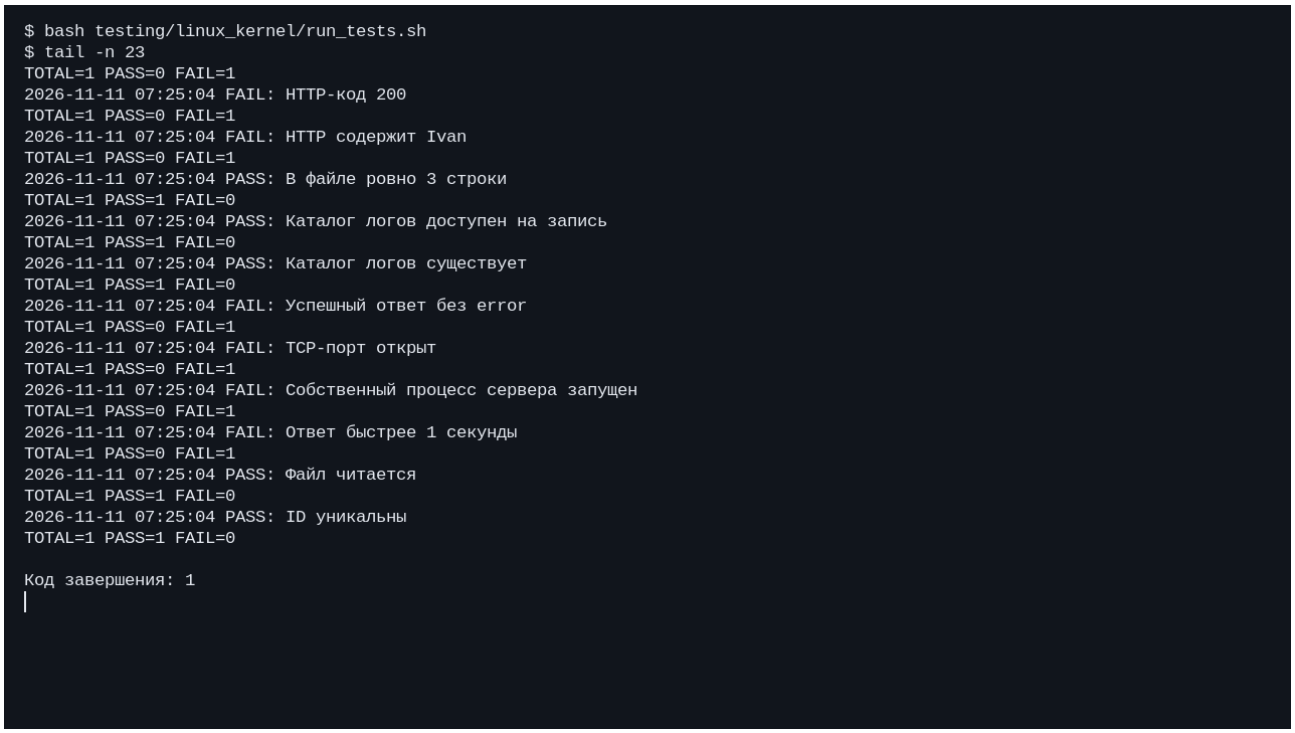


Рисунок 2 – Ошибки проверок при недоступном HTTP-сервисе

Ответы на контрольные вопросы

1. Что такое код возврата и как его проверить?

Число, возвращаемое процессом оболочке: 0 — успех, ненулевое значение — ошибка. Сразу после команды выполните echo “$?”. В наших агрегаторах наличие хотя бы одного FAIL даёт exit 1.

2. Зачем shift в check?

Первый аргумент — название проверки. shift удаляет его из позиционных параметров; “$@” становится выполняемой командой с её аргументами.

3. Для чего timeout?

Ограничивает ожидание соединения. Без него недоступный адрес может задержать весь прогон на системный TCP-таймаут.

4. Файл существует и сервер отдаёт файл — одно и то же?

Нет. Файл может быть на диске при остановленном сервере, неправильном URL, запрете доступа или неверном корне публикации. Проверки диска и HTTP независимы.

5. Как организовать 100 проверок?

Разделить по подсистемам, вынести общую функцию и конфигурацию, задать стабильные ID, агрегировать ненулевые коды, добавить машинный отчёт и запуск в CI. Независимые проверки можно распараллелить с лимитом.

6. Почему логировать каждый тест?

Итоговый счётчик не объясняет место и время сбоя. Имя проверки, timestamp и диагностика позволяют воспроизвести ошибку и сравнить ретест.

Проверка результатов

Результат проверки показан на рисунке 3.



Рисунок 3 – Журнал фактических запусков cron

Заключение

Реализованы Bash-проверки файловых данных, сетевого доступа и времени ответа. Общий запуск сохраняет результат каждой проверки и возвращает ненулевой код при обнаружении ошибки. Для CSV дополнительно проверяются уникальные идентификаторы. Подготовлен периодический запуск через cron; примеры его фактических вызовов показаны на снимке.

Положительный прогон выполнен при доступном локальном сервисе. После его остановки сетевые проверки обнаружили сбой, тогда как независимые файловые проверки продолжили работу. Это позволяет отделять отказ сервиса от ошибок входных данных. Тайм-ауты ограничивают ожидание недоступного узла, а журнал и коды возврата делают набор пригодным для запуска без постоянного наблюдения пользователя.

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ И ИНТЕРНЕТ-ИСТОЧНИКОВ

1. Емелина, Е.И. Поддержка и тестирование программных модулей [Текст]: учебное пособие / Е.И. Емелина. – Москва: КНОРУС, 2025. – 267 с.
2. Жуков, Р.А. Язык программирования Python: практикум [Текст]: учебное пособие / Р.А. Жуков. – Москва: ИНФРА-М, 2026. – 216 с.
3. Старолетов, С.М. Основы тестирования и верификации программного обеспечения [Текст]: учебное пособие / С.М. Старолетов. – 3-е изд., стер. – Санкт-Петербург: Лань, 2023. – 344 с.
4. Фёдорова, Г.Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности [Текст]: учебное пособие / Г.Н. Фёдорова. – Москва: КУРС: ИНФРА-М, 2024. – 336 с.
5. Чернышев, С.А. Основы программирования на Python [Текст]: учебное пособие для вузов / С.А. Чернышев. – 2-е изд., перераб. и доп. – Москва: Юрайт, 2024. – 349 с.

Электронные ресурсы

6. Документация платформы тестирования API Postman и утилиты Newman [Электронный ресурс]: https://learning.postman.com/docs/ (дата обращения 10.11.2026).
7. Международный комитет по квалификации тестирования программного обеспечения (ISTQB) [Электронный ресурс]: https://www.istqb.org/ (дата обращения 10.11.2026).
8. Руководство по командной оболочке GNU Bash и системным утилитам Linux [Электронный ресурс]: https://www.gnu.org/software/bash/manual/ (дата обращения 10.11.2026).

Листинги программ

Testing/linux_kernel/check.sh

```
#!/usr/bin/env bash
set -u
ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
DATA="${QA_DATA:-$ROOT/data}"
RUN="${QA_RUN:-$ROOT/.logs}"
mkdir -p "$RUN"
LOG_FILE="$RUN/test_$(date +%Y%m%d_%H%M%S)_$$.log"
PASS=0
FAIL=0
check() {
    local name="$1"
    shift
    if "$@" >"$RUN/last_stdout.txt" 2>"$RUN/last_stderr.txt"; then
        echo "$(date '+%F %T') PASS: $name" | tee -a "$LOG_FILE"
        PASS=$((PASS + 1))
    else
        echo "$(date '+%F %T') FAIL: $name" | tee -a "$LOG_FILE" "$RUN/alerts.log"
        cat "$RUN/last_stderr.txt" >> "$LOG_FILE"
        FAIL=$((FAIL + 1))
    fi
}
finish() {
    echo "TOTAL=$((PASS + FAIL)) PASS=$PASS FAIL=$FAIL" | tee -a "$LOG_FILE"
    test "$FAIL" -eq 0
}
```

Testing/linux_kernel/data/users.csv

```
id,name,status
1,Ivan,active
2,Maria,inactive
```

Testing/linux_kernel/functions.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/check.sh"
URL="${QA_URL:-http://127.0.0.1:8080/users.csv}"
HOST="${QA_HOST:-127.0.0.1}"
PORT="${QA_PORT:-8080}"
fetch() { curl -fsS --max-time 3 "$URL"; }
file_rows() { test "$(wc -l < "$DATA/users.csv")" -eq 3; }
file_fields() { awk -F, 'NF!=3{bad=1} END{exit bad}' "$DATA/users.csv"; }
unique_ids() { test -z "$(tail -n +2 "$DATA/users.csv" | cut -d, -f1 | sort | uniq -
d)"; }
http_code() { test "$(curl -s -o /dev/null -w '%{http_code}' --max-time 3 "$URL")" =
200; }
http_content() { local body; body="$(fetch)" || return 1; [[ "$body" == *Ivan* ]]; }
no_errors() { local body; body="$(fetch)" || return 1; ! printf '%s' "$body" | grep -qi
error; }
process_alive() { test -n "${QA_PID:-}" && kill -0 "$QA_PID" 2>/dev/null; }
port_open() { timeout 2 bash -c 'exec 3<>/dev/tcp/$1/$2' bash "$HOST" "$PORT"; }
```

```
response_fast() { local seconds; seconds="$(curl -fsS -o /dev/null -w '%{time_total}'
--max-time 3 "$URL")" || return 1; awk -v t="$seconds" 'BEGIN{exit !(t<1)}'; }
```

Testing/linux_kernel/run_all.sh

```
#!/usr/bin/env bash
ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
result=0
for script in "$ROOT"/test_*.sh; do
    bash "$script" || result=1
done
exit "$result"
```

Testing/linux_kernel/run_tests.sh

```
#!/usr/bin/env bash
exec bash "$(dirname "${BASH_SOURCE[0]}")/run_all.sh" "$@"
```

Testing/linux_kernel/test_content.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Заголовок CSV" grep -q "^id,name,status$" "$DATA/users.csv"
finish
```

Testing/linux_kernel/test_csv_format.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Каждая строка содержит 3 поля" file_fields
finish
```

Testing/linux_kernel/test_file.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Файл существует и не пуст" test -s "$DATA/users.csv"
finish
```

Testing/linux_kernel/test_http.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "HTTP доступен" fetch
finish
```

Testing/linux_kernel/test_http_code.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "HTTP-код 200" http_code
finish
```

Testing/linux_kernel/test_http_content.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "HTTP содержит Ivan" http_content
finish
```

Testing/linux_kernel/test_lines.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "В файле ровно 3 строки" file_rows
finish
```

Testing/linux_kernel/test_log_writable.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Каталог логов доступен на запись" test -w "$RUN"
finish
```

Testing/linux_kernel/test_logs.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Каталог логов существует" test -d "$RUN"
finish
```

Testing/linux_kernel/test_no_errors.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Успешный ответ без error" no_errors
finish
```

Testing/linux_kernel/test_port.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "TCP-порт открыт" port_open
finish
```

Testing/linux_kernel/test_process.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Собственный процесс сервера запущен" process_alive
finish
```

Testing/linux_kernel/test_response_time.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
```

```
check "Ответ быстрее 1 секунды" response_fast
finish
```

Testing/linux_kernel/test_rights.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Файл читается" test -r "$DATA/users.csv"
finish
```

Testing/linux_kernel/test_unique_ids.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "ID уникальны" unique_ids
finish
```